# 3. TShark

## Task 1 - Introduction

TShark is an open-source command-line network traffic analyser. It is created by the Wireshark developers and has most of the features of Wireshark. It is commonly used as a command-line version of Wireshark. However, it can also be used like tcpdump. Therefore it is preferred for comprehensive packet assessments.

## Task 2 - Command-Line Packet Analysis Hints | TShark and Supplemental CLI Tools

| Tool/Utility | Purpose and Benefit |
|---|---|
| **capinfos** | A program that provides details of a specified capture file. It is suggested to view the summary of the capture file before starting an investigation. |
| **grep** | Helps search plain-text data. |
| **cut** | Helps cut parts of lines from a specified data source. |
| **uniq** | Filters repeated lines/values. |
| **nl** | Views the number of shown lines. |
| **sed** | A stream editor. |
| **awk** | Scripting language that helps pattern search and processing. |

```
user@ubuntu$ capinfos demo.pcapng
File name:           demo.pcapng
File type:           Wireshark/tcpdump/... - pcap
File encapsulation:  Ethernet
```

View the details of the demo.pcapng file with "capinfos".

> What is the "RIPEMD160" value?

capinfos demo.pcapng RIPEMD160:
6ef5f0c165a1db4a3cad3116b0c5bcc0cf6b9ab7

## Task 3 - TShark Fundamentals I | Main Parameters I

| Parameter | Purpose |
|---|---|
| -h | • Display the help page with the most common features.<br>• `tshark -h` |
| -v | • Show version info.<br>• `tshark -v` |
| -D | • List available sniffing interfaces.<br>• `tshark -D` |
| -i | • Choose an interface to capture live traffic.<br>• `tshark -i 1`<br>• `tshark -i ens55` |
| **No Parameter** | • Sniff the traffic like tcpdump.<br>• `tshark` |

```
user@ubuntu$ tshark -v
TShark (Wireshark) 3 (Git v3. packaged as 3.)
```

**Sniffing:** TShark uses that interface to sniff the traffic. TShark will use the first available interface when no interface is selected, usually listed as 1 in the terminal.

> What is the installed TShark version in the given VM? tshark -v > 3.2.3

List the available interfaces with TShark. What is the number of available interfaces in the given VM? tshark -D > 12

## Task 4 - TShark Fundamentals I | Main Parameters II

| Parameter | Purpose |
|---|---|
| -r | • Read/input function. Read a capture file.<br>• `tshark -r demo.pcapng` |
| -c | • Packet count. Stop after capturing a specified number of packets.<br>• E.g. stop after capturing/filtering/reading 10 packets.<br>• `tshark -c 10` |

| | |
|---|---|
| -w | • Write/output function. Write the sniffed traffic to a file.<br>• `tshark -w sample-capture.pcap` |
| -V | • Verbose.<br>• Provide detailed information **for each packet**. This option will provide details similar to Wireshark's "Packet Details Pane".<br>• `tshark -V` |
| -q | • Silent mode.<br>• Suspress the packet outputs on the terminal.<br>• `tshark -q` |
| -x | • Display packet bytes.<br>• Show packet details in hex and ASCII dump for each packet.<br>• `tshark -x` |

**Read Capture Files:** TShark can also process PCAP files. You can use the `-r` parameter to process the file and investigate the packets. You can limit the number of shown packets using the `-c` parameter.

```
user@ubuntu$ tshark -r demo.pcapng
```

**Write Data:** TShark can also write the sniffed or filtered packets to a file. You can save the sniffed traffic to a file using the `-w` parameter. This option helps analysts to separate specific packets from the file/traffic and save them for further analysis.

```
user@ubuntu$ tshark -r demo.pcapng -c 1 -w write-demo.pcap
```

**Show Packet Bytes:** TShark can show packet details in hex and ASCII format. You can view the dump of the packets by using the `-x` parameter. Once you use this parameter, all packets will be shown in hex and ASCII format.

```
user@ubuntu$ tshark -r write-demo.pcap
```

**Verbosity:** Default TShark packet processing and sniffing operations provide a single line of information and exclude verbosity. The default approach makes it

easy to follow the number of processed/sniffed packets; however, TShark can also provide verbosity for each packet when instructed.

> Read the "demo.pcapng" file with TShark. What are the assigned TCP flags in the 29th packet?

**tshark -r demo.pcapng -c 29 > PSH, ACK**

> What is the "Ack" value of the 25th packet? **12421**

> What is the "Window size value" of the 9th packet? **9660**

## Task 5 - TShark Fundamentals II | Capture Conditions

Capture condition parameters only work in the “capturing/sniffing” mode. You will receive an error message if you try to read a pcap file and apply the capture condition parameters. The idea is to save the capture files in specific sizes for different purposes during live capturing. If you need to extract sorts of packets from a specific capture file, you will need to use the read&write options discussed in the previous task.

TShark supports combining autostop (`-a`) parameters with ring buffer control parameters (`-b`). You can combine the parameters according to your needs. Use the infinite loop options carefully; remember, you must use at least one autostop parameter to stop the infinite loop.

| Parameter | Purpose |
|---|---|
| | Define capture conditions for a single run/loop. STOP after completing the condition. Also known as "Autostop". |
| -a | • **Duration:** Sniff the traffic and stop after X seconds. Create a new file and write output to it.<br>◦ `tshark -w test.pcap -a duration:1`<br>• **Filesize:** Define the maximum capture file size. Stop after reaching X file size (KB).<br>◦ `tshark -w test.pcap -a filesize:10`<br>• **Files:** Define the maximum number of output files. Stop after X files.<br>◦ `tshark -w test.pcap -a filesize:10 -a files:3` |

| | |
|---|---|
| | Ring buffer control options. Define capture conditions for multiple runs/loops. (INFINITE LOOP). |
| -b | • **Duration:** Sniff the traffic for X seconds, create a new file and write output to it.<br>◦ `tshark -w test.pcap -b duration:1`<br>• **Filesize:** Define the maximum capture file size. Create a new file and write output to it after reaching filesize X (KB).<br>◦ `tshark -w test.pcap -b filesize:10`<br>• **Files:** Define the maximum number of output files. Rewrite the first/oldest file after creating X files.<br>◦ `tshark -w test.pcap -b filesize:10 -b files:3` |

> Which parameter can help analysts to create a **continuous** capture dump? -b

> Can we combine autostop and ring buffer parameters with TShark? y

## Task 6 - TShark Fundamentals III | Packet Filtering Options: Capture vs. Display Filters

There are two dimensions of packet filtering in TShark; live (capture) and post-capture (display) filtering. These two dimensions can be filtered with two different approaches; using a predefined syntax or Berkeley Packet Filters (BPF). TShark supports both, so you can use Wireshark filters and BPF to filter traffic.

| | |
|---|---|
| **Capture Filters** | Live filtering options. The purpose is to **save** only a specific part of the traffic. It is set before capturing traffic and is not changeable during live capture. |
| **Display Filters** | Post-capture filtering options. The purpose is to investigate packets by **reducing the number of visible packets**, which is changeable during the investigation. |

Capture filters are used to have a specific type of traffic in the capture file rather than having everything. Capture filters have limited filtering features, and the purpose is to implement a scope by range, protocol, and direction filtering. This might sound like bulk/raw filtering, but it still provides organised capture files with reasonable file size. The display filters investigate the capture files in-depth without modifying the packet.

| Parameter | Purpose |
|---|---|

| | |
|---|---|
| -f | Capture filters. Same as BPF syntax and Wireshark's capture filters. |
| -Y | Display filters. Same as **Wireshark's display filters.** |

## Task 7 - TShark Fundamentals IV | Packet Filtering Options: Capture Filters

Wireshark's capture filter syntax is used here. The basic syntax for the Capture/BPF filter is shown below. You can read more on capture filter syntax here(opens in new tab) and here(opens in new tab). Boolean operators can also be used in both types of filters.

The Berkeley Packet Filter (BPF) is a technology used in certain computer operating systems for programs that need to, among other things, analyze network traffic. BPF supports filtering packets, allowing a userspace process to supply a filter program that specifies which packets it wants to receive.

| Qualifier | Details and Available Options |
|---|---|
| **Type** | Target match type. You can filter IP addresses, hostnames, IP ranges, and port numbers. Note that if you don't set a qualifier, the "host" qualifier will be used by default.<br>• host \| net \| port \| portrange<br>• Filtering a host<br>◦ `tshark -f "host 10.10.10.10"`<br>• Filtering a network range<br>◦ `tshark -f "net 10.10.10.0/24"`<br>• Filtering a Port<br>◦ `tshark -f "port 80"`<br>• Filtering a port range<br>◦ `tshark -f "portrange 80-100"` |
| **Direction** | Target direction/flow. Note that if you don't use the direction operator, it will be equal to "either" and cover both directions.<br>• src \| dst<br>• Filtering source address<br>◦ `tshark -f "src host 10.10.10.10"`<br>• Filtering destination address<br>◦ `tshark -f "dst host 10.10.10.10"` |
| **Protocol** | Target protocol.<br>• arp \| ether \| icmp \| ip \| ip6 \| tcp \| udp |

- Filtering TCP
  - `tshark -f "tcp"`
- Filtering MAC address
  - `tshark -f "ether host F8:DB:C5:A2:5D:81"`
- You can also filter protocols with IP Protocol numbers assigned by IANA.
- Filtering IP Protocols 1 (ICMP)
  - `tshark -f "ip proto 1"`
  - **Assigned Internet Protocol Numbers**

We need to create traffic noise to test and simulate capture filters. We will use the "terminator" terminal instance to have a split-screen view in a single terminal. The "terminator" will help you craft and sniff packets using a single terminal interface. Now, run the `terminator` command and follow the instructions using the new terminal instance.

- First, run the given TShark command in Terminal-1 to start sniffing traffic.
- Then, run the given cURL command in Terminal-2 to create network noise.
- View sniffed packets results in Terminal-1.

"Terminator" Terminal Emulator Application Terminal-1;

```
user@ubuntu$ tshark -f "host 10.10.10.10"
Capturing on 'ens5'
    1 0.000000000 YOUR-IP → 10.10.10.10  TCP 74 36150 → 80 [S
YN] Seq=0 Win=62727 Len=0 MSS=8961 SACK_PERM=1
```

```
user@ubuntu$ curl -v 10.10.10.10 Trying 10.10.10.10:80...
* TCP_NODELAY set
* Connected to 10.10.10.10 (10.10.10.10) port 80 (#0)
> GET / HTTP/1.1
> Host: 10.10.10.10
> User-Agent: curl/7.68.0
```

Being comfortable with the command line and TShark filters requires time and practice. You can use the below table to practice TShark capture filters.

| Capture Filter Category | Details |
| --- | --- |
| **Host Filtering** | Capturing traffic to or from a specific host.<br>• Traffic generation with cURL. This command sends a default HTTP query to a specified address.<br>◦ `curl tryhackme.com`<br>• TShark capture filter for a host<br>◦ `tshark -f "host tryhackme.com"` |
| **IP Filtering** | Capturing traffic to or from a specific port. We will use the Netcat tool to create noise on specific ports.<br>• Traffic generation with Netcat. Here Netcat is instructed to provide details (verbosity), and timeout is set to 5 seconds.<br>◦ `nc 10.10.10.10 4444 -vw 5`<br>• TShark capture filter for specific IP address<br>◦ `tshark -f "host 10.10.10.10"` |
| **Port Filtering** | Capturing traffic to or from a specific port. We will use the Netcat tool to create noise on specific ports.<br>• Traffic generation with Netcat. Here Netcat is instructed to provide details (verbosity), and timeout is set to 5 seconds.<br>◦ `nc 10.10.10.10 4444 -vw 5`<br>• TShark capture filter for port 4444<br>◦ `tshark -f "port 4444"` |
| **Protocol Filtering** | Capturing traffic to or from a specific protocol. We will use the Netcat tool to create noise on specific ports.<br>• Traffic generation with Netcat. Here Netcat is instructed to use UDP, provide details (verbosity), and timeout is set to 5 seconds.<br>◦ `nc -u 10.10.10.10 4444 -vw 5`<br>• TShark capture filter for<br>◦ `tshark -f "udp"` |

> What is the number of packets with SYN bytes? 2

**terminal 1 tshark -f “host 10.10.10.10”** now on the **terminal 2** run this command **curl -v 10.10.10.10** now the packets gets captured on the terminal 1 and now you will need to count the number of SYN packets you will find the answer

`curl` — tool for making network requests (HTTP, HTTPS, FTP, etc.) This command downloads a website's integrity report to the terminal.

`v` — “verbose”; shows detailed connection/debug information

`10.10.10.10` — the target host/IP

> What is the number of packets sent to the IP address "10.10.10.10"? 7

To find the answer count the number of packets to 10.10.10.10 you will need to manually count the packets from the previous results

> What is the number of packets with ACK bytes? 8

To find the answer count the number of ACK packets on the given results

## Task 8 - TShark Fundamentals V | Packet Filtering Options: Display Filters

| Display Filter Category | Details and Available Options |
|---|---|
| **Protocol: IP** | • Filtering an IP without specifying a direction.<br>◦ `tshark -Y 'ip.addr == 10.10.10.10'`<br>• Filtering a network range<br>◦ `tshark -Y 'ip.addr == 10.10.10.0/24'`<br>• Filtering a source IP<br>◦ `tshark -Y 'ip.src == 10.10.10.10'`<br>• Filtering a destination IP<br>◦ `tshark -Y 'ip.dst == 10.10.10.10'` |
| **Protocol: TCP** | • Filtering TCP port<br>◦ `tshark -Y 'tcp.port == 80'`<br>• Filtering source TCP port<br>◦ `tshark -Y 'tcp.srcport == 80'` |
| **Protocol: HTTP** | • Filtering HTTP packets<br>◦ `tshark -Y 'http'`<br>• Filtering HTTP packets with response code "200"<br>◦ `tshark -Y "http.response.code == 200"` |
| **Protocol: DNS** | • Filtering DNS packets<br>◦ `tshark -Y 'dns'`<br>• Filtering all DNS "A" packets<br>◦ `tshark -Y 'dns.qry.type == 1'` |

Sample filtering query: The below terminal demonstrates using the "IP filtering" option. TShark filters the packets and provides the output in our terminal.

```
user@ubuntu$ tshark -r demo.pcapng -Y 'ip.addr == 145.253.2.2
03'
13 2.55 145.254.160.237 ? 145.253.2.203 DNS Standard query 0x
0023 A ..
17 2.91 145.253.2.203 ? 145.254.160.237 DNS Standard query re
sponse 0x0023 A ..
```

```
user@ubuntu$ tshark -r demo.pcapng -Y 'http'
4   0.911 145.254.160.237 ? 65.208.228.223 HTTP GET /downloa
d.html HTTP/1.1
18   2.984 145.254.160.237 ? 216.239.59.99 HTTP GET /pagead/a
ds?client...
27   3.955 216.239.59.99 ? 145.254.160.237 HTTP HTTP/1.1 200
OK  (text/html)
38   4.846 65.208.228.223 ? 145.254.160.237 HTTP/XML HTTP/1.1
200 OK
```

You can use the `nl` command to get a numbered list of your output. Therefore you can easily calculate the "total number of filtered packets" without being confused with "assigned packet numbers". The usage of the `nl` command is shown below.

```
user@ubuntu$ tshark -r demo.pcapng -Y 'http' | nl 1
4  0.911 145.254.160.237 ? 65.208.228.223 HTTP GET /download.
html HTTP/1.1
2   18  2.984 145.254.160.237 ? 216.239.59.99 HTTP GET /pagea
d/ads?client...
3   27   3.955 216.239.59.99 ? 145.254.160.237 HTTP HTTP/1.1
200 OK (text/html)
4   38   4.846 65.208.228.223 ? 145.254.160.237 HTTP/XML HTT
P/1.1 200 OK
```

**Use the "demo.pcapng" file to answer the questions.**

> What is the number of packets with a "65.208.228.223" IP address? 34

**tshark -r demo.pcapng -Y “ip.addr == 65.208.228.223” | wc -l**

> What is the number of packets with a "TCP port 3371"? 7

**tshark -r demo.pcapng -Y “tcp.port == 3371” | wc -l**

> What is the number of packets with a "145.254.160.237" IP address as a source address? 20

**tshark -r demo.pcapng -Y “ip.src == 145.254.160.237” | wc -l**

> **Rerun the previous query and look at the output.** What is the packet number of the "Duplicate" packet? 37

## Task 9 - Conclusion

In this room, we covered TShark, what it is, how it operates, and how to use it to investigate traffic captures.